UET.CS2043 - Lập trình nâng cao

Unit Testing and Test Automation

Outline

- Introduction to Software Testing
  - Focus on unit testing
- Test design
  - Equivalence partitioning
  - Boundary value analysis
  - T-way combinatorial testing
- Test execution
  - Test automation with JUnit
- Best practices
- Exercises

What is software testing?

- Software testing is a systematic process of executing and evaluating software to:
  - Detect defects and unexpected behavior
  - Verify compliance with specified requirements
  - Reduce the risk of failures in production

Testing levels

- Software testing is performed at multiple levels, progressing from individual components to the complete system
  - Unit testing
    - Tests individual units or components in isolation
  - Integration testing
    - Tests interactions between combined units
  - System testing
    - Tests the entire integrated system as a whole
  - Acceptance testing
    - Validates the system from the user or business perspective

Unit testing

- Verify the correctness of the “smallest” testable unit of software
  - Tested unit: typically a single class or individual methods
- Units are tested independently of other components
  - Interactions with external classes or services are mocked or stubbed
- Test inputs: Method invocations with controlled parameters
- Test oracle: assertions on
  - Returned values
  - Object state
  - Thrown exceptions

Test case

- A test case is a well-defined specification that describes how to verify the correct behavior of a software unit
- A test case consists of:
  - Inputs
    - Data or method calls provided to the unit under test
  - Execution conditions
    - The required system state before execution
  - Expected results (test oracle)
    - The expected output used to determine correctness

Test activities

- Test design:
  - Identify what to test and how to test it
  - Derive test cases and expected outcomes
- Test execution:
  - Execute the test cases
  - Observe and interpret results

How to design a test case?

- Identify the testing target
  - Determine the unit to be tested (e.g., methods, classes)
  - Analyze its specification
- Identify representative input values
- Generate test cases

Identify representative input values

- Black-box testing techniques:
  - Focus on specifications, not implementation
    - Equivalence partitioning
    - Boundary value analysis
    - Random testing
    - Error guessing
- White-box testing techniques:
  - Focus on internal structure and code
    - Derive inputs based on the program's control flow and data flow

Equivalence partitioning

- Partition the input space into groups of values (equivalence classes)
- Values within the same class should produce similar behavior
- Select one representative value from each class for testing

Boundary value analysis

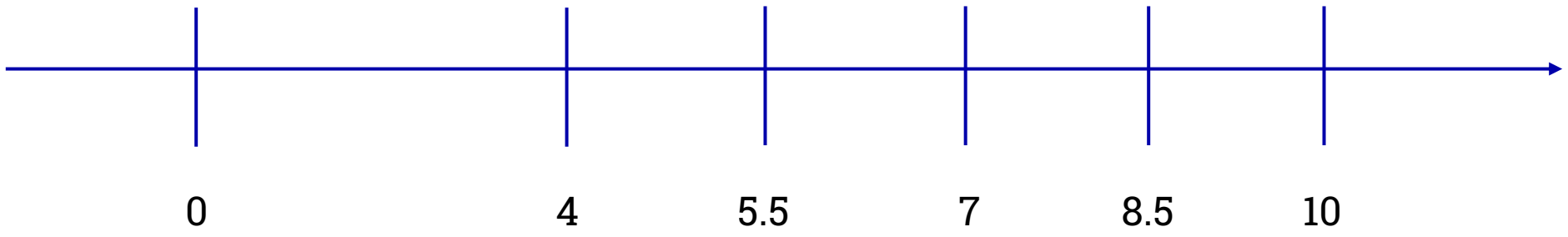
- Focuses on testing values at the boundaries of the input ranges or equivalence classes
- Selection rules:
  - For an input in range [min, max] test:
    - min-, min, min+
    - nominal value (nom)
    - max-, max, max+

Example

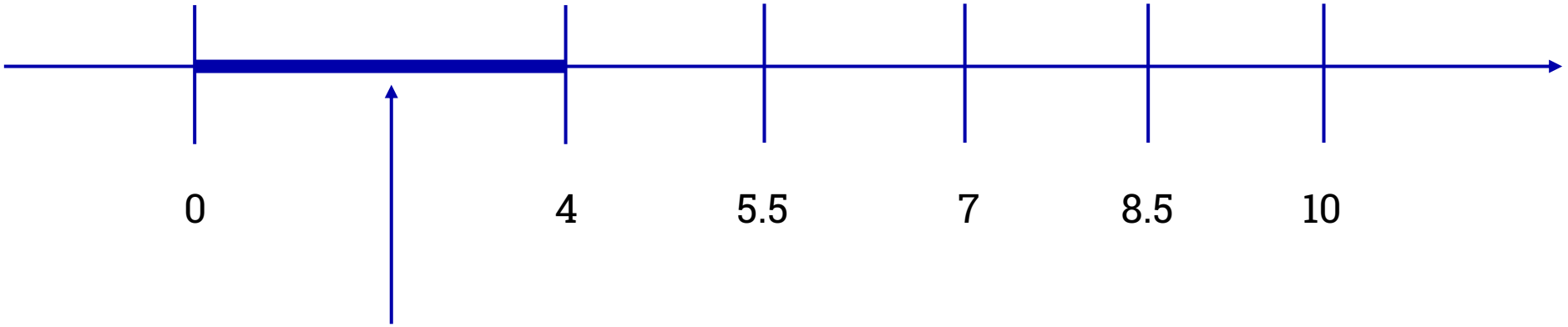
- Problem: Grade conversion method that maps the final assessment scores to grades based on predefined score interval:
- $score \in [0, 10]$

Score range	Grade
0 – 3.9	F (Fail)
4.0 – 5.4	D (Poor)
5.5 – 6.9	C (Satisfactory)
7.0 – 8.4	B (Good)
8.5 – 10	A (Excellent)

Example: Equivalence partitioning

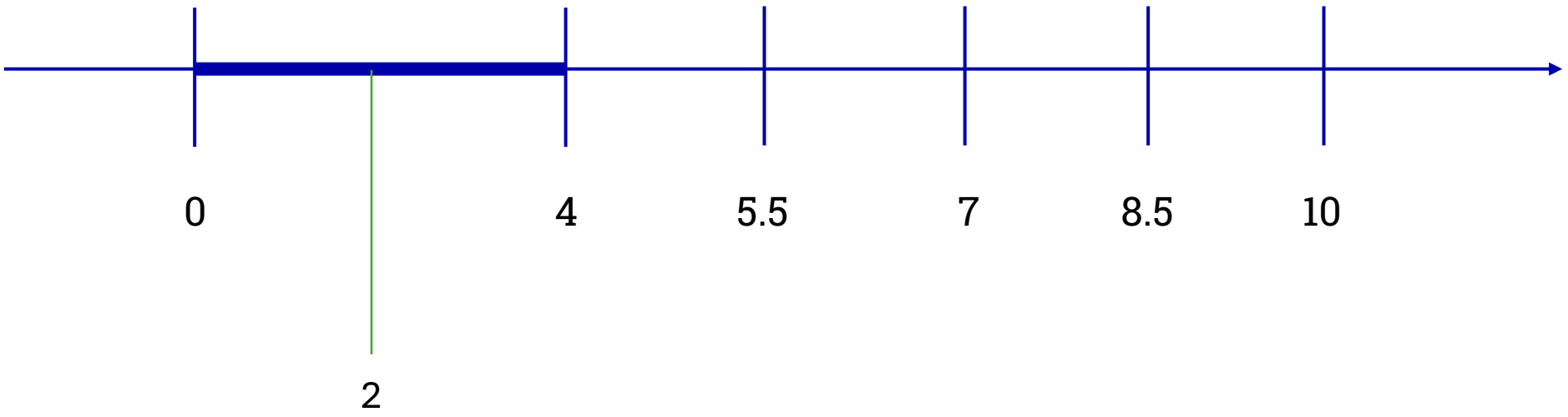


Example: Equivalence partitioning



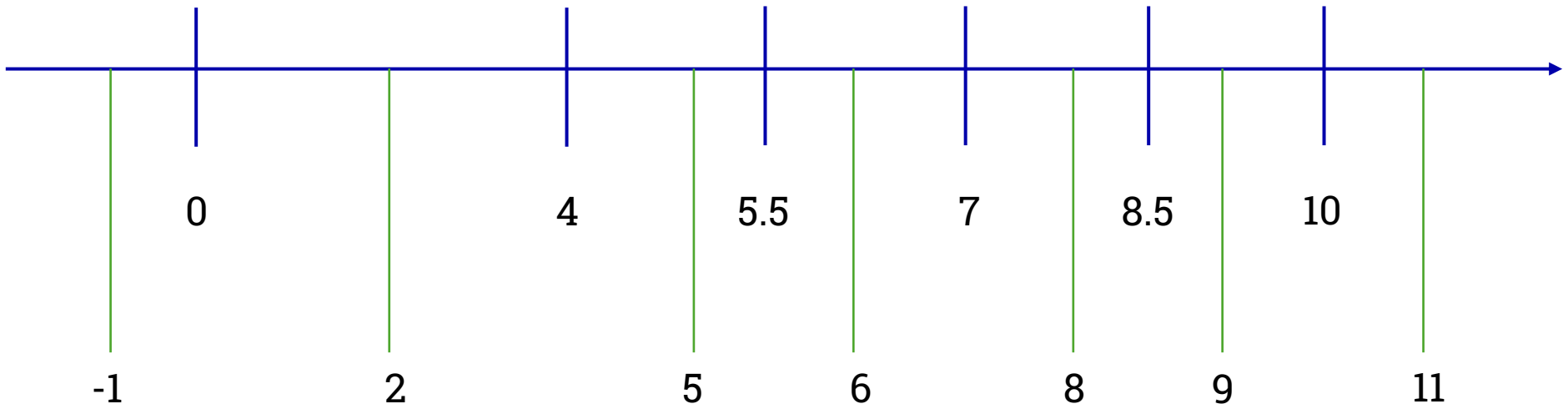
The interval [0, 3.9] forms an equivalence class, as all values in this range are mapped to grade F.

Example: Equivalence partitioning



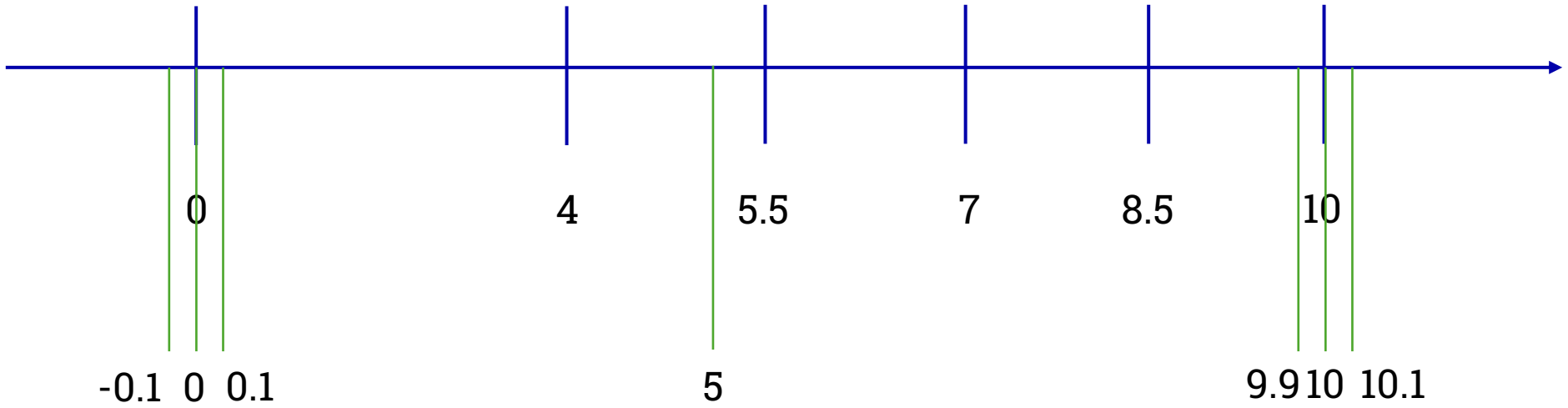
Select a representative value from this equivalence class, e.g., 2

Example: Equivalence partitioning



Example: Boundary value analysis

- Domain boundaries specify the lower and upper limits of the valid input range for a variable



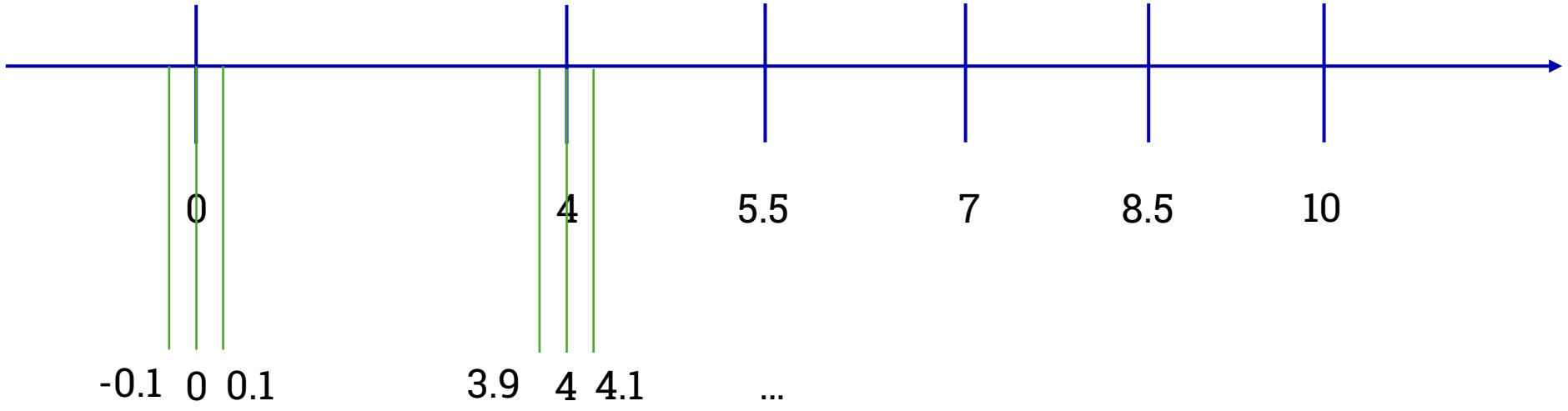
min- = -0.1
min = 0
min+ = 0.1

nom = 5

max- = 9.9
max = 10
min+ = 10.1

Example: Boundary value analysis

- Internal boundaries are the boundary points between adjacent equivalence classes within the valid input domain.



Combinations of input parameters

- When a testing unit has multiple input parameters, and each parameter can take multiple possible values

→ How should we combine parameter values to systematically create effective test cases?

T-way combinatorial testing

- Ensures possible combination of values of any t input parameters appears in at least one test case
- Balances test coverage and test suite size

What does “t-way” mean?

- Consider a method with parameters: P_1, P_2, ..., P_n where each parameter has a set of values for testing
  - 1-way: each value of every parameter is used in at least one test case
  - 2-way (pairwise): Every possible combination of values of any two parameters appears together in at least one test case.
  - 3-way: Every possible combination of values of any three parameters appears together in at least one test case.
  - n-way: exhaustive testing, every possible combination of parameter values is tested

Example

- Given a method with three parameter:
- `processOrder(userType, paymentMethod, DeliveryType)`
- Each parameter has the following test values:
  - `userType: {Guest, Member}`
  - `paymentMethod: {Cash, Card}`
  - `deliveryType: {Standard, Express}`

Example: 1-way testing

Test case	`userType`	`paymentMethod`	`deliveryType`
TC1	Guest	Cash	Standard
TC2	Member	Card	Express

Each value of every parameter appears in at least one test case

Example: 2-way testing

Test case	`userType`	`paymentMethod`	`deliveryType`
TC1	`Guest`	`Cash`	`Standard`
TC2	`Guest`	`Card`	`Express`
TC3	`Member`	`Cash`	`Express`
TC4	`Member`	`Card`	`Standard`

All possible combinations of values for every pair of parameters, (`userType, paymentMethod`), (`paymentMethod, deliveryType`), (`userType, deliveryType`), are covered.

Test execution

- How do we run test cases on a program?
  - Test cases can be executed manually by running the program and checking the results.
  - However, manual execution should be avoided
    - Humans are slow, expensive, and error-prone
  - Test design requires effort and creativity
  - Test execution should be automated whenever possible

Test automation

- Automates repetitive test execution tasks
- Preserves human effort for creative test design
- Enables controlled execution and environments
- Supports automatic result checking and test re-execution

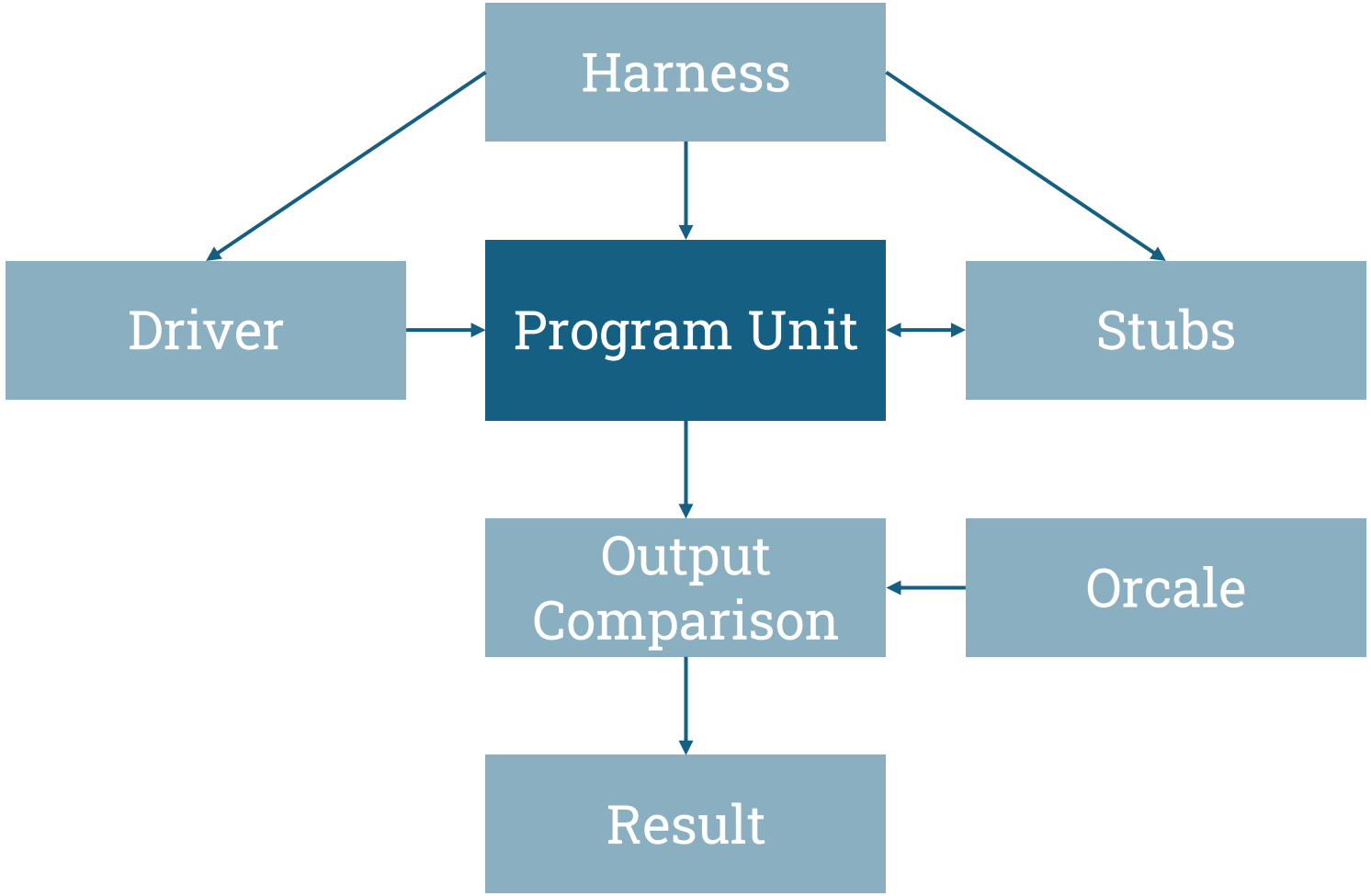
Test scaffolding

- Test scaffolding is a set of auxiliary programs and artifacts created to support test automation
- Not part of the system
- Often temporary, used only during testing
- Test scaffolding enables:
  - Testing before all system components are completed
  - Testing individual components in isolation
  - Control over the test environment and conditions

Testing scaffolding

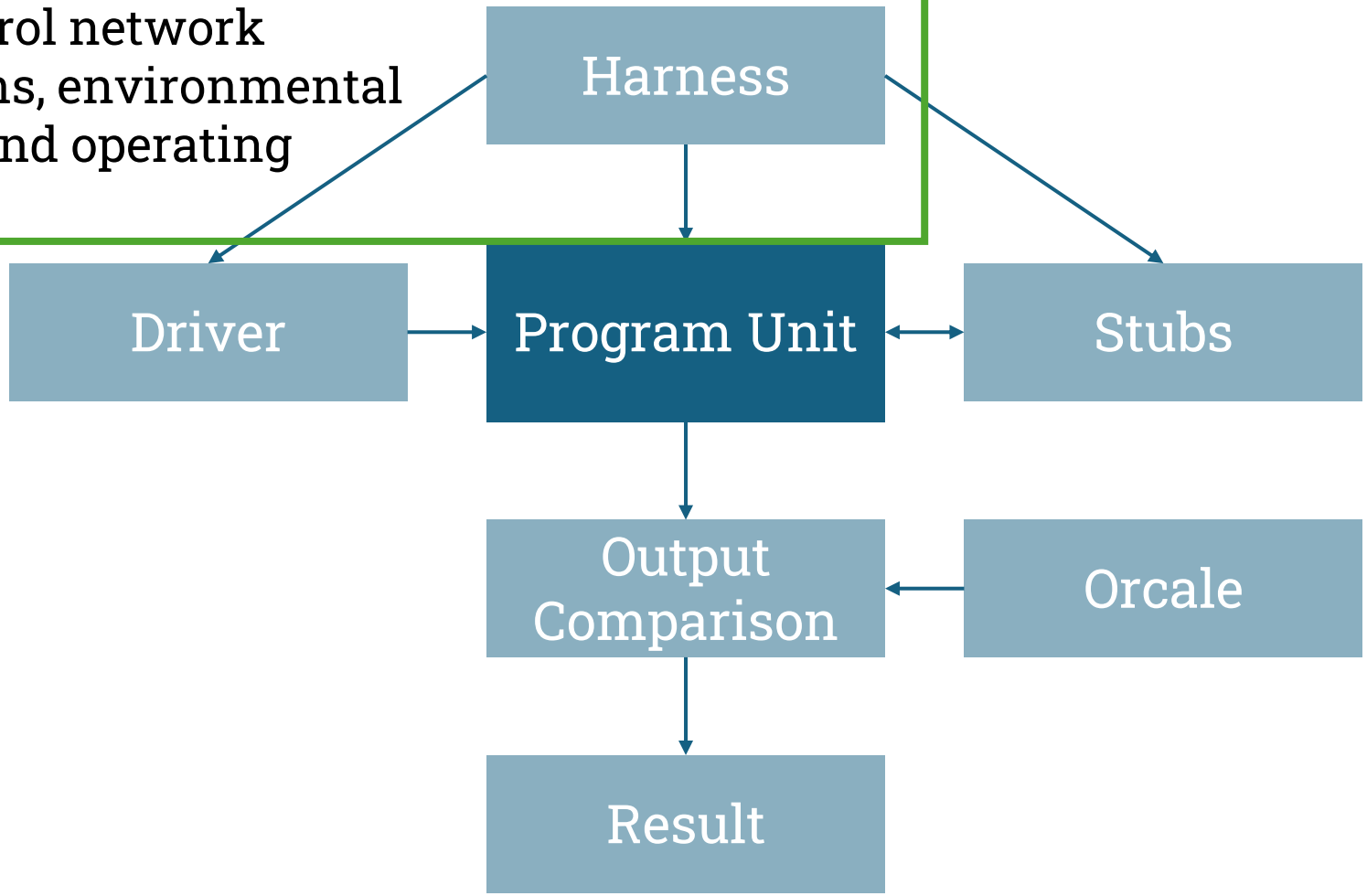
- Drivers: simulate calling programs
- Test harness: simulates the execution environment
- Stubs/mocks: replace missing or external components
- Execution support: manage and record test runs

Testing scaffolding

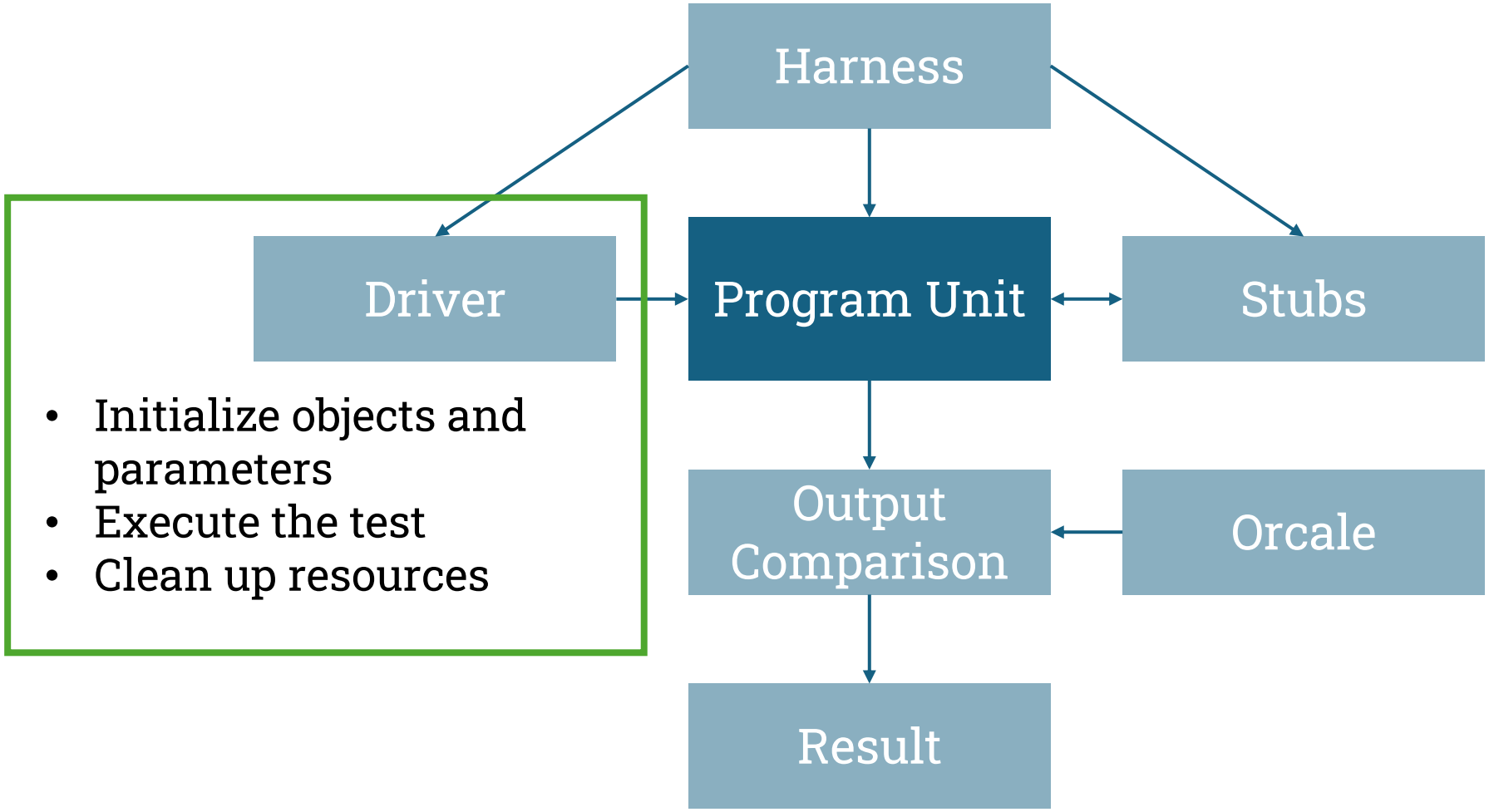


Testing scaffolding

- Simulates the execution environment
- Can control network conditions, environmental factors, and operating systems

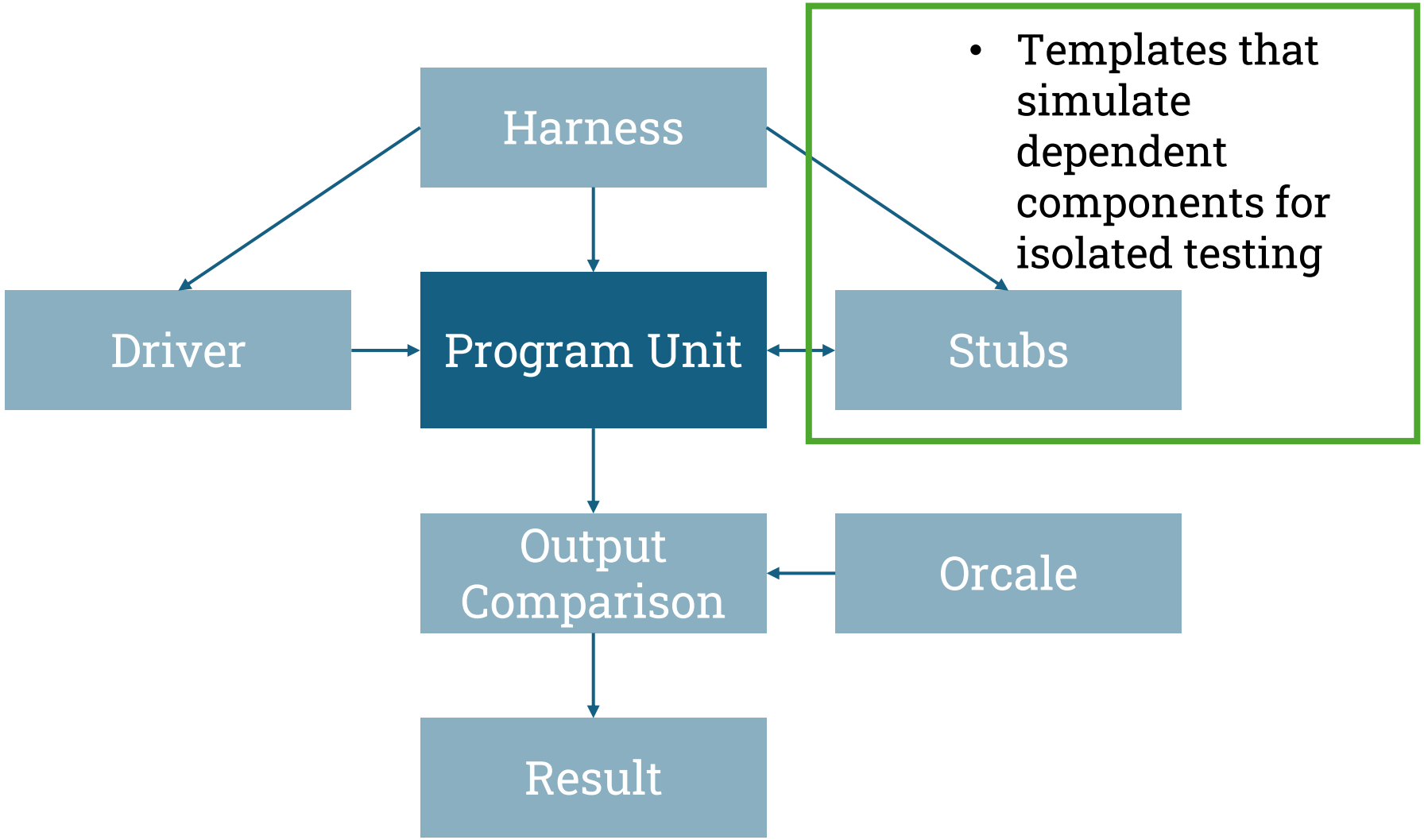


Testing scaffolding



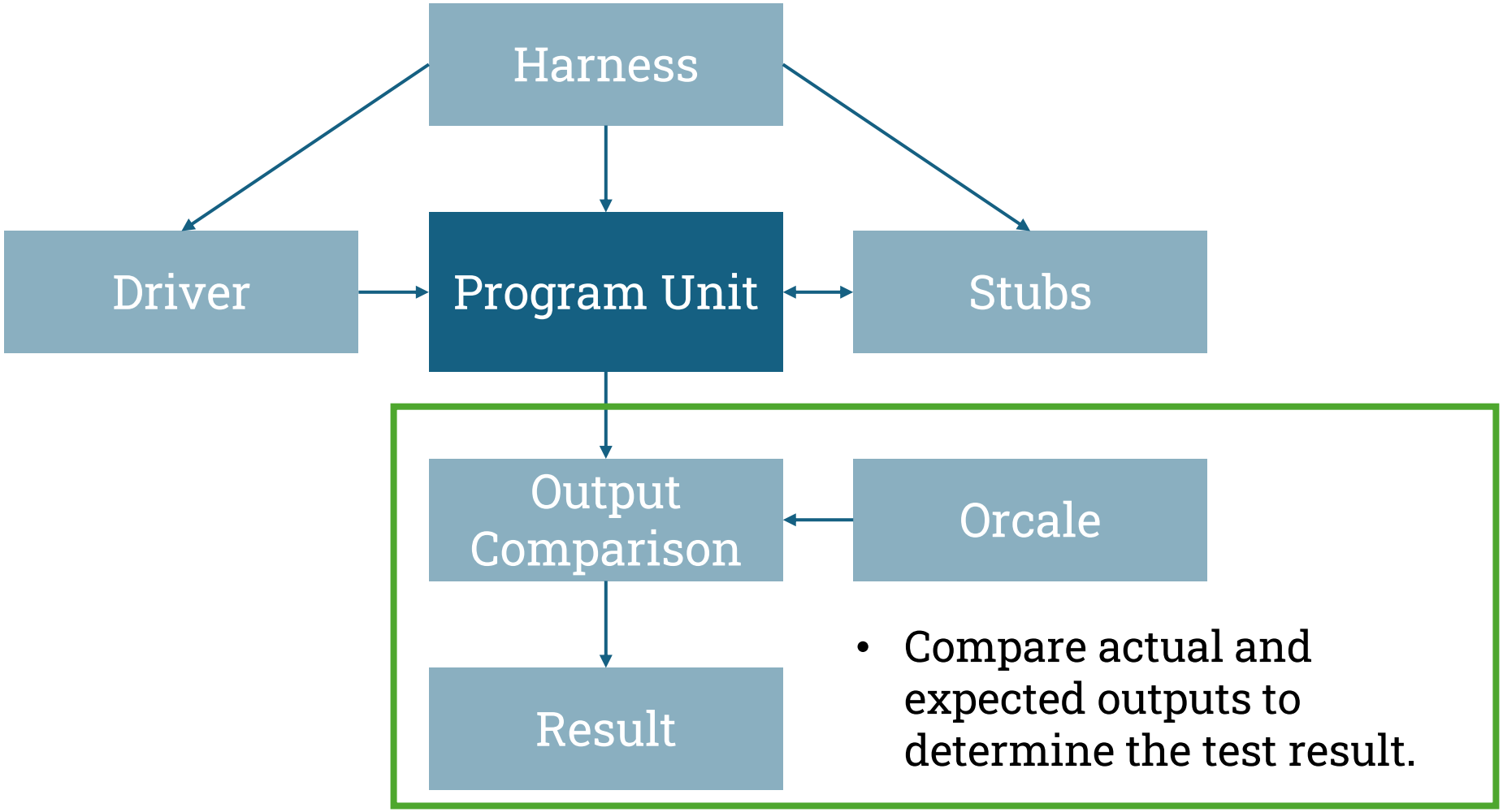
- Initialize objects and parameters
- Execute the test
- Clean up resources

Testing scaffolding



- Templates that simulate dependent components for isolated testing

Testing scaffolding



- Compare actual and expected outputs to determine the test result.

Writing an executable test case

- Test input
  - All required input data
- Expected output (test oracle)
  - What should happen, i.e., values or exceptions
- Initialization
  - Any steps that must be taken before test execution
- Test steps
  - Interactions with the unit under test (e.g., method calls), and comparisons of actual and expected outputs
- Tear down
  - Any steps that must be taken after test execution

JUnit

- Java framework for automated unit testing
- Uses assertions to check expected results
- Supports test lifecycle management
- Integrates with IDEs and build tools

Example: Calculator class

```java
public class Calculator {

    public int add(int a, int b) {
        return a + b;
    }

    public int divide(int a, int b) {
        if (b == 0) {
            throw new IllegalArgumentException("Division by zero");
        }
        return a / b;
    }
}
```

Example: CalculatorTest class

```java
import org.junit.Test;
import static org.junit.Assert.assertEquals;
import static org.junit.Assert.assertThrows;


public class CalculatorTest {

    @Test
    public void testAdd() {
        Calculator calc = new Calculator();
        int result = calc.add( a: 2,  b: 3);
        assertEquals( expected: 5, result);
    }

    @Test
    public void testDivide() {
        Calculator calc = new Calcu
        assertEquals( expected: 2, ca
    }

    @Test
    public void testDivideByZero() -
        Calculator calc = new Calcu
        assertThrows(IllegalArgumentException.class,
                () -> calc.divide( a: 5,  b: 0));
    }
}
```

Annotation `@test`: marks a test method

CalculatorTest	99 ms
testDivideByZero	99 ms
testAdd	0 ms
testDivide	0 ms

Common JUnit annotations

- Shared Initialization:
  - `@BeforeEach`: runs before each test
  - `@BeforeAll`: runs once before all tests

- Tear down:
  - `@AfterEach`: runs after each test
  - `@AfterAll`: runs ones after all tests

Example

```java
public class CalculatorTest {
    static Calculator calc = new Calculator();

    @BeforeAll
    static void setUpBeforeAll() {
        System.out.println("BeforeAll: Initialize shared resources");
        Calculator calc = new Calculator();
    }

    @Test
    public void testAdd() {
        int result = calc.add( a: 2,  b: 3);
        assertEquals( expected: 5, result);
    }

    @Test
    public void testDivide() {
        assertEquals( expected: 2, calc.divide( a: 6,  b: 3));
    }

    @Test
    public void testDivideByZero() {
        assertThrows(IllegalArgumentException.class,
                () -> calc.divide( a: 5,  b: 0));
    }
}
```

`setUpBeforeAll` is executed once before all tests and is typically used to initialize static object calc

Assertions

- Assertions are “language” of testing
- They express contrainsts that must hold for a test to pass by comparing expected and actual outputs

Common assertions

- Value equality
  - `assertEquals(expected, actual)`
  - `assertArrayEquals(expected, actual)`
- Boolean conditions
  - `assertTrue(condition)`
  - `assertFalse(condition)`
- Null checks
  - `assertNull(object)`
  - `assertNotNull(object)`

Common assertions

- Reference equality
  - `assertSame(expected, actual)`
  - `assertNotSame(expected, actual)`
- Flexible/expressive assertions
  - `assertThat(actual, matcher)`

Testing exceptions

- When testing error handling, we expect certain exceptions to be thrown under exceptional conditions
  - `assertThrows`: check whether the code block throws the expected exception
  - `assertEquals`: can be used to check the content of the stack trace

Testing performance

- Verify execution time constraints
- `assertTimeout` enforces a time limit on an action

```
36      @Test
37      void add_shouldFinishWithinTimeLimit() {
38          assertTimeout(Duration.ofMillis(5), () -> {
39              int result = calc.add( a: 10,  b: 20);
40              assertEquals( expected: 30, result);
41          });
42      }
43  }
```

Best practices

- Use assertions instead of print statements
- If code is non-deterministic, tests should give deterministic results

```java
public long calculateTime(){
    long time = 0;
    long before = System.currentTimeMillis();
    complexFunction();
    long after = System.currentTimeMillis();
    time = after - before;
    return time;
}
```

Tests for this method should not specify exact time, but properties of a "good" execution:

- The time should be positive, not negative or 0
- A range on the allowed times

Exercises

Ex1

- Given class `MathUtils`, write the test cases for the following methods: `max(int a, int b)` and `divide(int a, int b)`
- `a` and `b` are integers in the range `[Integer.MIN_VALUE, Integer.MAX_VALUE]`.
- For `divide(int a, int b)`, make sure your test suite includes cases where exceptions are expected (e.g., division by zero).
- Design your test cases using both Equivalence Partitioning (EP) and Boundary Value Analysis (BVA).

```java
public class MathUtils {

    /**
     * Returns the greater of two integers.
     *
     * @param a the first integer
     * @param b the second integer
     * @return the maximum value between {@code a} and {@code b}
     */
    public static int max(int a, int b) {
        if (a >= b) return a;
        return b;
    }

    /**
     * Divides one integer by another.
     *
     * @param a the dividend
     * @param b the divisor
     * @return the result of {@code a / b}
     * @throws IllegalArgumentException if {@code b == 0}
     */
    public static int divide(int a, int b) {
        if (b == 0) {
            throw new IllegalArgumentException("Divider must not be zero");
        }
        return a / b;
    }
}
```

Ex2

Write test suites to test at least 2 programs that you have already implemented in the course.

For each selected class and its key methods:

1. Design test cases using EP
2. Design test cases using BVA

EX3

- Given this problem: https://www.hackerrank.com/challenges/java-biginteger/problem?isFullScreen=true
- Implement the solution
- Write a test class to test the implemented methods

EX4

- Given this problem: https://www.hackerrank.com/challenges/java-stack/problem?isFullScreen=true
- Implement the solution
- Write a test class to test the implemented methods

EX5

- Given this problem: https://www.hackerrank.com/challenges/java-date-and-time/problem?isFullScreen=true
- Implement the solution
- Write a test class to test the implemented methods